

# Road Lane Boundary Detection Using Classical Image Analysis Techniques

Apurv Chokshi  
Jarvis College of Computing and Digital Media  
DePaul University  
Chicago, USA  
achoksh2@depaul.edu

**Abstract**— Lane detection is a fundamental component of autonomous driving and advanced driver assistance systems (ADAS), as it allows vehicles to understand road structure and maintain safe positioning within driving lanes. This project presents a classical image analysis pipeline for detecting road lane boundaries from images captured by a forward-facing vehicle camera. The proposed system processes input images through several stages, including grayscale conversion, Gaussian smoothing, gradient-based edge detection using the Canny operator, color-based segmentation, and region of interest masking to isolate the road area. The Hough Line Transform is then applied to detect line segments corresponding to lane boundaries, which are subsequently overlaid on the original image. Experimental results demonstrate that the approach can successfully detect lane boundaries in structured road scenes with clear markings and stable lighting conditions. However, performance is sensitive to challenging conditions such as shadows, faded lane markings, and curved roads. The project highlights both the strengths and limitations of classical image analysis methods for lane detection.

**Keywords**—Brain Lane Detection, Image Analysis, Edge Detection, Hough Transform, Autonomous Driving

## I. INTRODUCTION

Lane detection is an essential component of modern autonomous driving systems and advanced driver assistance systems (ADAS). These technologies rely on accurate perception of the road environment to ensure safe navigation and maintain proper vehicle positioning within traffic lanes. By identifying lane boundaries from camera images, a vehicle can better understand road geometry and support functions such as lane keeping, lane departure warning, and automated steering assistance. As autonomous driving technologies continue to advance, reliable lane detection remains a critical computer vision problem that directly impacts vehicle safety and navigation performance.

The task of lane detection involves extracting meaningful road structures from raw visual data captured by forward-facing cameras. However, this problem is challenging because real-world road scenes often contain varying lighting conditions, shadows, worn lane markings, and complex backgrounds such as buildings, sidewalks, and vehicles. These factors can make it difficult to accurately identify lane boundaries using simple visual cues. As a result, robust lane detection requires carefully designed image processing techniques that can isolate relevant road features while suppressing noise and irrelevant information.

In this project, a classical image analysis pipeline is developed to detect road lane boundaries from forward-facing camera images. The proposed approach processes input images through several stages of image processing and analysis. First, the image is converted to grayscale and smoothed using Gaussian filtering to reduce noise. Gradient-based edge detection is then applied using the Canny edge detector to identify strong intensity transitions corresponding to lane boundaries. Color-based segmentation is also used to highlight bright lane markings. A region of interest mask is applied to isolate the road area, and the Hough Line Transform is used to detect line segments representing lane boundaries. The final step overlays the detected lanes onto the original image to visualize the detection results.

Preliminary results show that the proposed pipeline can successfully detect lane boundaries in structured road scenes with clearly visible lane markings and stable lighting conditions. The combination of gradient-based edge detection and color segmentation improves the robustness of the detection process, while the region of interest mask helps reduce noise from irrelevant parts of the scene. However, the system remains

sensitive to challenging conditions such as shadows, faded markings, and road curvature. These findings highlight both the effectiveness and the limitations of classical image analysis techniques for lane detection tasks.

## II. BACKGROUND

Lane detection has been widely studied in the field of computer vision due to its importance in autonomous driving and advanced driver assistance systems (ADAS). Reliable detection of road lane boundaries allows vehicles to maintain proper positioning within lanes and supports functionalities such as lane departure warning and lane keeping assistance. In most practical systems, lane detection relies on images captured by forward-facing cameras mounted on vehicles. These images must be analyzed to extract meaningful geometric structures that correspond to lane boundaries on the road surface.

Early approaches to lane detection primarily relied on classical image processing techniques. These methods typically use gradient-based edge detection, color segmentation, and geometric line detection to identify lane markings. For example, edge detection algorithms such as the Canny operator are commonly used to detect strong intensity gradients that correspond to lane boundaries. Region-based techniques, including color thresholding and region of interest masking, are used to isolate the road area and reduce noise from irrelevant image regions such as buildings or the sky. After extracting candidate edge pixels, geometric techniques such as the Hough Line Transform are often applied to detect line segments corresponding to lane markings. While these classical approaches are computationally efficient and interpretable, their performance can be sensitive to variations in lighting conditions, shadows, worn lane markings, and complex road geometries.

More recently, research has explored data-driven and deep learning approaches for lane detection. These methods aim to overcome the limitations of classical techniques by learning more robust representations of lane structures directly from training data. For example, Jin et al. (2022) proposed the Eigenlanes framework, which represents lane shapes using a set of basis vectors derived from training data through singular value decomposition. This approach allows lane boundaries to be represented as combinations of learned lane shape components, improving detection performance across structurally diverse lane configurations. Similarly, the RCLane method models lane boundaries as chains of connected points and predicts lane structure using deep neural networks. This approach enables the detection of complex lane geometries such as curved lanes, forks, and merges that are difficult for traditional line-based methods to capture.

Despite the strong performance of modern learning-based methods, classical image analysis techniques remain useful for understanding the fundamental principles of lane detection and building interpretable vision pipelines. In this project, a classical image processing approach is implemented to detect lane boundaries from forward-facing road images. By combining

edge detection, color segmentation, region of interest masking, and Hough-based line detection, the system demonstrates how meaningful road structures can be extracted from raw pixel data. This approach provides insight into the strengths and limitations of classical methods while highlighting the challenges that motivate more advanced approaches in current research.

## III. METHODOLOGY

This project implements a classical image analysis pipeline to detect road lane boundaries from images captured by a forward-facing camera mounted on a vehicle. The goal of the pipeline is to transform raw pixel data into meaningful geometric structures representing lane boundaries. The overall system consists of several stages, including image preprocessing, edge detection, color-based segmentation, region-of-interest masking, and geometric line detection. The final step overlays the detected lane lines onto the original image for visualization.

The implementation was developed in Python using Google Colab. Several widely used computer vision libraries were used to support the implementation. The primary library is OpenCV, which provides functions for image processing operations such as grayscale conversion, Gaussian smoothing, edge detection, color space transformations, and Hough line detection. NumPy was used for numerical operations and array manipulation, as images are represented as multidimensional matrices of pixel values. Additionally, Matplotlib was used to visualize intermediate and final outputs within the notebook environment. These libraries provide efficient implementations of many standard computer vision operations and significantly simplify the development of the image processing pipeline.

The overall lane detection pipeline consists of a sequence of image processing steps that progressively refine the input image to isolate lane boundaries. The pipeline is illustrated in Figure 1.

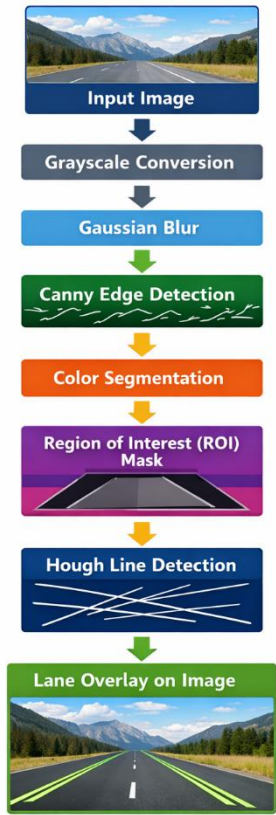


Figure 1 : Pipeline Diagram

#### A. Image Preprocessing

The first stage of the pipeline converts the input image into grayscale using the OpenCV function `cv2.cvtColor`. Converting the image to grayscale simplifies the data by removing color information and focusing only on intensity variations, which are important for detecting edges.

Next, Gaussian smoothing is applied using the `cv2.GaussianBlur` function. Gaussian filtering reduces high-frequency noise in the image while preserving major structures such as lane markings. This step improves the stability of subsequent edge detection by preventing small noise patterns from producing false edges.

#### B. Edge Detection

After preprocessing, Canny edge detection is applied to identify strong intensity gradients in the image. The Canny detector is implemented using the OpenCV function `cv2.Canny`. This algorithm detects edges by identifying pixels where the intensity gradient exceeds certain thresholds. Lane boundaries typically produce strong gradients because lane markings are brighter than the surrounding asphalt surface.

The resulting edge map highlights potential lane boundaries as well as other structural edges present in the image.

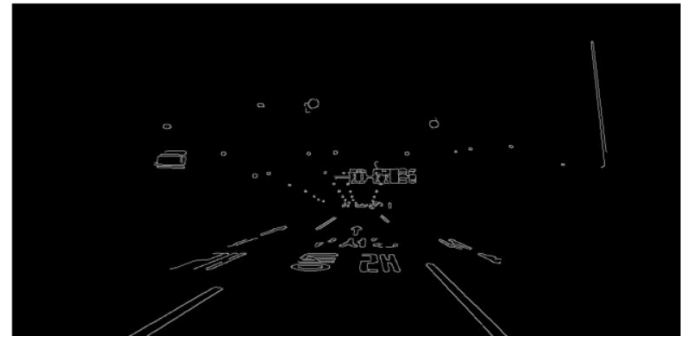


Figure 2 : Edge Detection in action

#### C. Color-Based Segmentation

In addition to gradient-based edge detection, the pipeline also uses color-based segmentation to detect lane markings. Lane lines are typically white or yellow, making them distinguishable from the darker road surface.

To perform segmentation, the image is converted to the HLS color space using OpenCV. Threshold values are then applied to isolate pixels corresponding to bright white or yellow lane markings. This produces a binary mask that highlights candidate lane pixels. The segmentation mask is combined with the edge detection output to improve the reliability of lane detection.

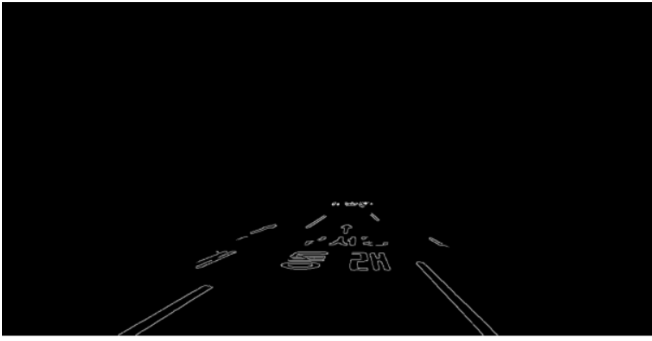


Figure 3 : Combined Edge and Color Mask

#### D. Region of Interest Selection

Real-world driving scenes contain many edges and structures that are not related to the road. To reduce noise, a region of interest (ROI) mask is applied to restrict the analysis to the road area.

The ROI is defined as a trapezoidal region corresponding to the expected location of the road surface in the image. Since the camera is assumed to be mounted at a fixed position on the vehicle, the road region appears in approximately the same location in each frame. The ROI mask is implemented using OpenCV functions such as `cv2.fillPoly` and `cv2.bitwise_and`. This step removes irrelevant regions such as buildings, sidewalks, and the sky, which significantly improves the accuracy of lane detection.



*Figure 4 : you can figure out the trapezoid boundaries here. That is our region of interest*

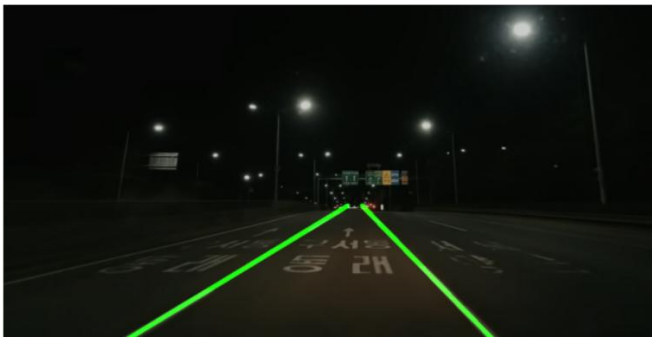
#### E. Hough Line Detection

After identifying candidate edge pixels within the region of interest, the system applies the Hough Line Transform to detect line segments corresponding to lane boundaries. The probabilistic Hough transform is implemented using the OpenCV function `cv2.HoughLinesP`.

This method converts edge pixels into a parameter space where lines can be detected based on the alignment of edge points. The detected line segments are then filtered and averaged to estimate the final lane boundaries. The resulting lane lines are drawn onto the original image using OpenCV drawing functions to produce the final visualization.

#### F. Final Visualization

The final step of the pipeline overlays the detected lane boundaries onto the original image using the OpenCV function `cv2.addWeighted`. This produces an output image in which the detected lanes are highlighted, allowing easy visualization of the lane detection results.



*Figure 5 : Final Result that highlights the lane boudaries*

### IV. RESULTS

About the performance of the proposed lane detection pipeline using a set of test images, The goal of these experiments is to analyze how well the system detects lane boundaries under different road conditions and to identify both successful detections and failure cases. Since this project focuses on

understanding classical image analysis techniques rather than developing a fully optimized system, the evaluation is primarily qualitative. However, several experiments were performed to observe how different stages of the pipeline contribute to the final result.

The experiments were conducted using a small collection of road images representing the viewpoint of a forward-facing vehicle camera. The images were obtained from publicly available road scenes and additional images collected from online sources. I even took screenshots from YouTube videos. All images follow a similar perspective, simulating the typical camera placement used in autonomous driving systems and advanced driver assistance systems (ADAS).

The selected test images include a variety of road conditions in order to evaluate the robustness of the pipeline. These include scenes with clearly visible lane markings, mild shadows, urban environments with surrounding buildings, and cases where lane markings are partially faded. This variety helps evaluate how well the classical image analysis pipeline performs under both favorable and challenging conditions.

The evaluation focuses on observing the effectiveness of each stage of the image analysis pipeline. Intermediate outputs were examined to verify that each processing step contributes to improving the final detection results. These intermediate outputs include grayscale images, edge maps generated by the Canny edge detector, color segmentation masks, and region-of-interest filtered images.

In addition to analyzing intermediate outputs, the final results were evaluated by visually comparing the detected lane boundaries with the true lane markings in the input images. Successful detections were identified when the algorithm correctly detected lane boundaries and aligned them with the visible lane markings on the road. Failure cases were also analyzed to understand the conditions under which the algorithm performs poorly.

In scenes where lane markings are clearly visible and lighting conditions are stable, the proposed pipeline performs well. The combination of gradient-based edge detection and color segmentation allows the algorithm to identify candidate lane pixels effectively. The region of interest mask helps remove irrelevant edges from surrounding structures such as buildings and sidewalks. As a result, the Hough Line Transform is able to detect line segments corresponding to lane boundaries with good accuracy.

Figure 6 shows an example of a successful detection case. In this example, the system correctly identifies the lane boundary and overlays the detected lane line onto the original image. The detected line closely follows the visible lane marking on the road

surface, demonstrating the effectiveness of the pipeline under favorable conditions.

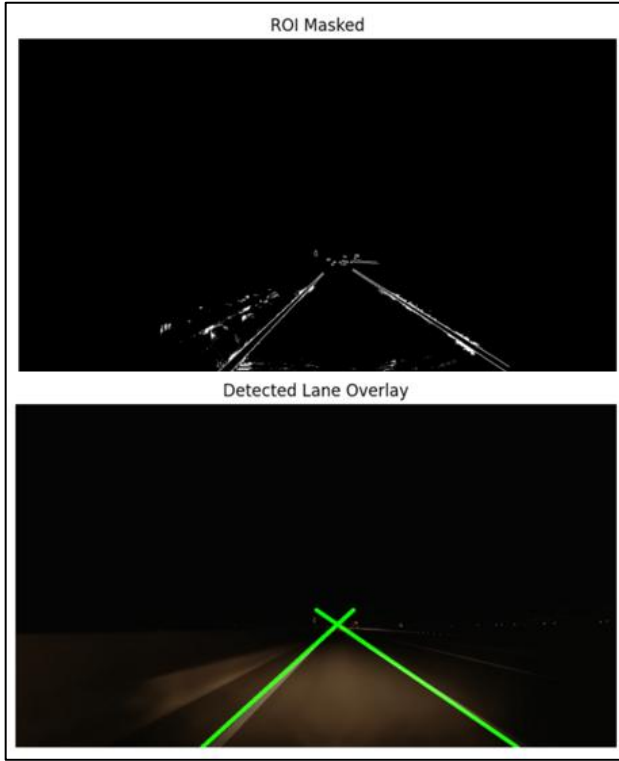


Figure 6

Despite its success in structured road scenes, the classical lane detection pipeline also exhibits limitations. Several failure cases were observed during testing. These typically occur in scenarios where lane markings are partially faded, where strong shadows are present, or where complex road structures introduce additional edges. It is a Rainy weather in this example

For example, in some images the algorithm detects incomplete lane segments or incorrectly identifies other edges in the scene as potential lane boundaries. This occurs because classical edge-based methods rely heavily on gradient information and can be sensitive to lighting changes and noise in the image. Figure 7 shows an example of a failure case where the detected lane does not fully align with the actual lane marking due to these challenges.

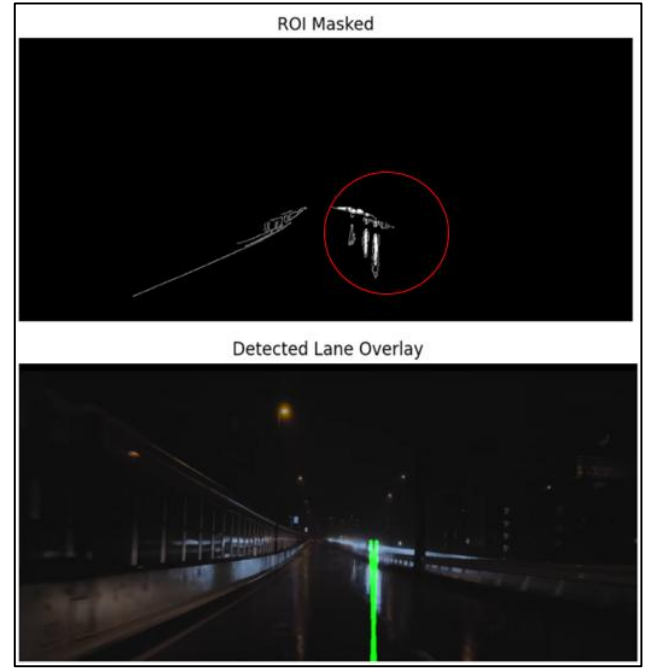


Figure 7

Overall, the experimental results demonstrate that the proposed classical image analysis pipeline can successfully detect lane boundaries in many structured road environments. The integration of edge detection, color segmentation, and region-of-interest masking improves the reliability of lane detection compared to using a single technique alone.

However, the results also highlight the limitations of classical image processing approaches when dealing with complex real-world scenarios. Environmental factors such as shadows, faded markings, and road curvature can significantly affect detection performance. These observations emphasize the importance of robust feature extraction and explain why more advanced learning-based approaches have been explored in recent lane detection research.

## V. DISCUSSION

The experimental results demonstrate that the proposed classical image analysis pipeline is capable of detecting lane boundaries in structured road environments where lane markings are clearly visible and lighting conditions are relatively stable. The integration of multiple processing stages—including preprocessing, edge detection, color segmentation, region-of-interest masking, and geometric line detection—helps isolate lane features from the surrounding environment. However, the experiments also reveal several limitations of the approach, particularly when the visual characteristics of the road scene deviate from the assumptions made by the pipeline.

One important factor influencing performance is the selection of hyper-parameters used in different stages of the pipeline. For example, the Gaussian blur kernel size affects the balance between noise reduction and edge preservation. Smaller kernel sizes preserve fine image details but may allow noise to remain in the image, which can lead to spurious edges during the Canny edge detection stage. Larger kernel sizes reduce noise more effectively but can also weaken lane boundaries, making them more difficult to detect. Similarly, the Canny edge detection thresholds significantly influence the number of detected edges. If the thresholds are too low, many irrelevant edges appear in the edge map, which may confuse the Hough line detection stage. Conversely, if the thresholds are too high, important lane edges may be missed.

The region of interest (ROI) parameters also play an important role in the effectiveness of the system. Because the camera is assumed to be mounted in a fixed position on the vehicle, the road region appears in roughly the same location in each image. This assumption allows a fixed trapezoidal ROI mask to remove irrelevant portions of the scene such as buildings, sidewalks, and the sky. In most cases, this significantly improves detection performance by reducing the number of candidate edges considered during line detection. However, if the ROI is not properly aligned with the road region—for example in images with different camera perspectives—important lane information may be removed.

Several components of the system performed as expected. In particular, the combination of edge detection and color segmentation improved the reliability of lane detection compared to using edge detection alone. Edge detection identifies strong structural features, while color segmentation helps isolate lane markings that are typically brighter than the road surface. The Hough Line Transform also performed well in detecting straight lane segments when the input edge map was clean and the lane markings were clearly visible.

However, some limitations were observed in more challenging scenarios. In images with strong shadows, faded lane markings, or complex urban environments, the algorithm sometimes detected incorrect line segments or failed to detect complete lane boundaries. This occurs because classical edge-based methods rely heavily on strong intensity gradients, which may not always be present in real-world road scenes. Additionally, the Hough line detection stage assumes that lane boundaries can be approximated by straight lines, which may not be accurate for curved roads.

Several opportunities for improvement were identified based on these observations. One potential improvement would be to implement curve fitting techniques, such as polynomial regression, to better model curved lane boundaries. Another improvement would be to incorporate adaptive thresholding or more robust segmentation methods to improve performance under varying lighting conditions. Finally, temporal smoothing techniques could be applied when processing video sequences,

allowing the system to stabilize lane detections across consecutive frames.

Overall, the discussion highlights that while classical image analysis techniques can provide a useful baseline solution for lane detection, real-world driving environments present challenges that may require more advanced approaches to achieve consistent and reliable performance.

## VI. CONCLUSION

This project explored the use of classical image analysis techniques to detect road lane boundaries from images captured by a forward-facing vehicle camera. A complete image processing pipeline was implemented using several stages, including grayscale conversion, Gaussian smoothing, Canny edge detection, color-based segmentation, region-of-interest masking, and Hough line detection. The final output overlays the detected lane boundaries onto the original image, providing a visual representation of the lane detection results.

The experimental results show that classical image processing methods can effectively detect lane boundaries when lane markings are clearly visible and lighting conditions are relatively stable. The combination of gradient-based edge detection and color segmentation proved particularly useful for isolating lane markings from the surrounding road surface. Additionally, the region-of-interest masking step significantly improved performance by removing irrelevant portions of the scene and reducing noise during line detection.

However, the project also revealed several limitations of classical edge-based methods. The detection performance can degrade in challenging scenarios such as strong shadows, faded lane markings, and curved road segments. Since the Hough line detection method assumes straight lines, it may not accurately represent curved lane boundaries in more complex driving environments.

Despite these limitations, the project provided valuable insight into how meaningful road structures can be extracted from raw image data using classical computer vision techniques. One interesting aspect of this project was observing how different image processing stages interact to gradually refine the detection results. Future work could explore improvements such as curve fitting methods for curved lanes, adaptive thresholding techniques, and more robust segmentation approaches. These enhancements could improve the reliability of lane detection in more diverse and complex road environments.

Overall, this project demonstrates both the strengths and limitations of classical image analysis approaches for lane detection and highlights the challenges involved in designing robust computer vision systems for real-world autonomous driving applications.

## REFERENCES

- [1] Jin, H., Han, R., & Kim, J. (2022). *Eigenlanes: Data-driven lane descriptors for structurally diverse lanes*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
- [2] Zhang, Y., Wang, X., Xu, L., & Zhao, H. (2022). *RCLane: Relay chain prediction for lane detection*. arXiv preprint arXiv:2207.09399.
- [3] Bradski, G. (2000). *The OpenCV library*. Dr. Dobb's Journal of Software Tools.
- [4] OpenCV. (2024). *Open source computer vision library*.
- [5] NumPy Developers. (2024). *NumPy: The fundamental package for scientific computing with Python*.
- [6] Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. Computing in Science & Engineering, 9(3), 90–95.